



Java

Inhalt

1. Grundgerüst
2. Primitive Datentypen
3. Variablen und Konstanten
4. Operatoren
5. Casting (Umwandlung von Datentypen)
6. Datentyp String
7. Hilfsmittel für die heutige Übung

Grundgerüst

Grundgerüst

- Jedes Java Programm braucht einen klar definierten Einstiegspunkt

```
public class Programm {  
    public static void main(String args[]) {  
    }  
}
```

Primitive Datentypen

Primitive Datentypen

- Primitive Datentypen lassen sich nicht weiter in kleinere Datentypen zerlegen.
- Beginnen mit einem kleinen Buchstaben

Primitive Datentypen

- Wahrheitswerte
 - boolean
- Zeichen und Symbole
 - char
- ganze Zahlen
 - byte, short, int, long
- reelle Zahlen
 - float, double

Wertebereich

Datentyp	Wertebereich	BIT
boolean	true, false	undefiniert
char	16-Bit Unicode-Zeichensatz	16
byte	-128 bis 127	8
short	-32.768 bis 32.767	16
int	-2.147.483.648 bis 2.147.483.647	32
long	-9.223.372.036.854.775.808 bis 9.223.372.036.854.775.807	64
float	+/-1,4E-45 bis +/-3,4E+38	32
double	+/-4,9E-324 bis +/-1,7E+308	64

Variablen und Konstanten

Variablen - Deklaration

- Variablen dienen als Platzhalter für Werte im Speicher
- Um Variablen im Speicher anzulegen, müssen sie deklariert werden
- Namenskonvention:
 - erster Buchstabe klein
 - möglichst aussagekräftig, einzelne Buchstaben nur in kurzen Blöcken
 - i,j,k als Laufvariablen

■ Syntax:

- `<Datentyp> <Name>`

■ Beispiel:

- `boolean b;`
`char c;`
`int x;`

Variablen – Zuweisung und Initialisierung

- Nach der Deklaration kann der Variable ein Wert zugewiesen werden
- Die erste Zuweisung im Programm heißt Initialisierung

■ Syntax:

- `<Datentyp> <Name> = <Wert>`

■ Beispiel:

- `boolean b = true;`
`char c = 'z';`
`int x = 3;`

Konstanten

- Konstanten sind Variablen die nach der Zuweisung unverändert bleiben.
- Namenskonvention
 - komplett in Großbuchstaben
 - einzelne Wörter durch „_“ getrennt

- **Syntax:**

- `final <Datentyp> <Name> = <Wert>`

- **Beispiel:**

- `final boolean WAHR = true;`

Operatoren

Operatoren

- Logische Operatoren
 - Mit dem Datentyp boolean
- Arithmetische Operatoren
 - Mit den Datentypen byte, short, int, long, float und double
- Vergleichsoperatoren
 - Mit allen primitiven Datentypen

Logische Operatoren

Operator	Funktion
&&	Logisches UND
	Logisches ODER
!	Negation

■ Beispiel:

```
■ boolean a, b, c;  
  a = true, b = false;  
  c = !a || b;
```

Logische Operatoren

Logisches UND

- Mit dem logischen Operator UND ist das Ergebnis nur dann true, wenn alle Elemente darin true sind
- Bei welchen boolean a, b ergibt sich welches Ergebnis bei (a && b)?

a	b	a && b
true	true	
true	false	
false	true	
false	false	

Logische Operatoren

Logisches UND

- Mit dem logischen Operator UND ist das Ergebnis nur dann true, wenn alle Elemente darin true sind
- Bei welchen boolean a, b ergibt sich welches Ergebnis bei (a && b)?

a	b	a && b
true	true	true
true	false	false
false	true	false
false	false	false

Logische Operatoren

Logisches ODER

- Mit dem logischen Operator ODER ist das Ergebnis immer dann true, wenn wenigstens ein Element darin true ist
- Bei welchen boolean a, b ergibt sich welches Ergebnis bei (a || b)?

a	b	a b
true	true	
true	false	
false	true	
false	false	

Logische Operatoren

Logisches ODER

- Mit dem logischen Operator ODER ist das Ergebnis immer dann true, wenn wenigstens ein Element darin true ist
- Bei welchen boolean a, b ergibt sich welches Ergebnis bei (a || b)?

a	b	a b
true	true	true
true	false	true
false	true	true
false	false	false

Logische Operatoren

Negation

- Ein negierter boolean Wert ist dann true, wenn der Wert false ist
- Bei welchen boolean a, b ergibt sich welches Ergebnis bei !a und !b?

a	!a	b	!b
true		true	
true		false	
false		true	
false		false	

Logische Operatoren

Negation

- Ein negierter boolean Wert ist dann true, wenn der Wert false ist
- Bei welchen boolean a, b ergibt sich welches Ergebnis bei $(!a \ \&\& \ b)$, $(a \ \&\& \ !b)$ $(!a \ || \ b)$, $(a \ || \ !b)$?

a	!a	b	!b	!a && b	a && !b	!a b	a !b
true	false	true	false				
true	false	false	true				
false	true	true	false				
false	true	false	true				

Logische Operatoren

Negation

- Ein negierter boolean Wert ist dann true, wenn der Wert false ist
- Bei welchen boolean a, b ergibt sich welches Ergebnis bei (!a && b), (a && !b) (!a || b), (a || !b)?

a	!a	b	!b	!a && b	a && !b	!a b	a !b
true	false	true	false	false	false	true	true
true	false	false	true	false	true	false	true
false	true	true	false	true	false	true	false
false	true	false	true	false	false	true	true

Arithmetische Operatoren

Operator	Funktion
+	Addition
-	Subtraktion
/	Division
*	Multiplikation
%	Modulo (Rest bei Division)

■ Beispiel:

```
■ long x = 5, y = 2;  
  long z = (x / y) + (x % y) - 1;
```

Vergleichsoperatoren

Operator	Funktion
==	Gleich
!=	Ungleich
<	Kleiner
>	Größer
<=	Kleiner gleich
>=	Größer gleich

Vergleichsoperatoren - Beispiel

- Welche Ergebnisse ergeben sich bei den int a=5, b=7?

a == b	a != b	a < b	a > b	a <= b	a >= b

b == a	b != a	b < a	b > a	b <= a	b >= a

a == a	a != a	a < a	a > a	a <= a	a >= a

Vergleichsoperatoren - Beispiel

- Welche Ergebnisse ergeben sich bei den `int a=5, b=7`?

<code>a == b</code>	<code>a != b</code>	<code>a < b</code>	<code>a > b</code>	<code>a <= b</code>	<code>a >= b</code>
false	true	true	false	true	false

<code>b == a</code>	<code>b != a</code>	<code>b < a</code>	<code>b > a</code>	<code>b <= a</code>	<code>b >= a</code>
false	true	false	true	false	true

<code>a == a</code>	<code>a != a</code>	<code>a < a</code>	<code>a > a</code>	<code>a <= a</code>	<code>a >= a</code>
true	false	false	false	true	true

Operatorenreihenfolge

- Klammern ()
- Postfix Operatoren (a++)
- Unäre Operatoren (-a)
- Punktrechnung (*, /, %)
- Strichrechnung (+, -)
- Shift-Operatoren
- Relationale Operatoren (<, >, <=, >=, instanceof)
- Gleichheitsoperatoren (==, !=)
- Bitoperatoren
- Logische Operatoren (&&, ||)
- Zuweisungsoperatoren (=, +=, ...)

Casting (Umwandlung von Datentypen)

Typumwandlung

- Werte können in gleiche oder andere Datentypen kopiert werden
- Bei verschiedenen Datentypen muss gecastet werden
- Unterscheidung
 - Implizites Casting
 - Explizites Casting

Casting

Implizites Casting

- speichert einen Wert eines kleineren Datentyps in einen größeren
 - z.B. int in long oder float in double
- Typumwandlung ist unproblematisch

- **Beispiel:**

- `int x = 24;`
`long y;`
`y = x;`

Explizites Casting

- speichert einen Wert eines größeren Datentyps in einen kleineren
 - z.B. long in int oder double in float
- kann zu Datenverlust führen, wenn der Wert zu groß ist

- **Beispiel:**

- `long x = 24;`
`int y;`
`y = (int) x;`

Datentyp String

String

- Zusammengesetzter Datentyp
- Zeichenkette
- Vergleichsoperatoren sind nicht anwendbar
- Dafür Verwendung von String eigenen Methoden

- **Syntax:**

- `<Datentyp> <Name> = <Wert>`

- **Beispiel:**

- `String wort = "Wort";`
`String zeichen = "x";`
`String zahl = "24";`

String – wichtigste Methoden

Rückgabe	Name	Input	Funktion
char	charAt	int	Gibt den char an der gewünschten Stelle
String	substring	int, int	Teilstring zwischen den gewählten Stellen (zweites int ist die erste Stelle nach Substring)
String	trim		Gibt den String ohne Leerstellen
int	length		Gibt die Länge des Strings
boolean	equals	Object	Gibt an, ob der String den gleichen Inhalt hat
boolean	equalsIgnoreCase	String	Vernachlässigt Unterschiede in der Groß- und Kleinschreibung

String – wichtigste Methoden

Rückgabe	Name	Input	Funktion
boolean	startsWith	String	Überprüft ob String mit String anfängt
boolean	endsWith	String	Überprüft ob String mit String endet
int	indexOf	String	Gibt die Stelle vom String oder -1, wenn nicht vorhanden
String	toLowerCase		Gibt String in Kleinbuchstaben
String	toUpperCase		Gibt String in Großbuchstaben
String	replace	char, char	Gibt den String, in dem bestimmte Zeichen ersetzt wurden

String – Beispiel Methode

- Zwei Strings „string1“ und „string2“ haben bereits einen Wert zugewiesen bekommen. Nun soll überprüft werden, ob die beiden Strings den gleichen Inhalt haben. Das Ergebnis soll in den boolean „isIdentical“ gespeichert werden

String – Beispiel Methode

- Zwei Strings „string1“ und „string2“ haben bereits einen Wert zugewiesen bekommen. Nun soll überprüft werden, ob die beiden Strings den gleichen Inhalt haben. Das Ergebnis soll in den boolean „isIdentical“ gespeichert werden
- **Beispiel:**
 - `boolean isIdentical = string1.equals(string2);`

String – in Zahlen konvertieren

- Verwendung von zusammengesetzten Datentypen (Wrapperklassen) für Zahlen
- Beispiele:
 - `int z1 = Integer.parseInt(<String mit Zahlen als Inhalt>);`
 - `long z2 = Long.parseLong(<String mit Zahlen als Inhalt>);`



Wenn der String aus unzulässigen Zeichen besteht führt dies zu einem Fehler!

String – Aus Strings zusammensetzen

- Ein String kann aus mehreren Strings durch „+“ zusammengesetzt werden
- **Beispiele:**
 - `String string1 = "Dieser Satz " + "ist zusammengesetzt";`
 - `String s1 = "Wort";`
`String s2 = "Der String s1 hat den Wert "+s1;`

Ein- und Ausgabe

Werkzeuge für Heute

- Ausgaben
- Eingaben
- If - Bedingungen

Ausgaben

- Werte von Variablen u.Ä. können über die Konsole ausgegeben werden
- Dazu können benutzt werden:
 - `System.out.print(<hier Inhalt>);`
 - `System.out.println(<hier Inhalt>);`

Eingaben

- Verwendung von JOptionPane
- Eigentlich bereits ein Objekt für eine Oberfläche
- Für diesen Umfang einfache Verwendung dieses Schemas:
 - `String <Name der Variable für Eingabe> = JOptionPane.showInputDialog(null, <String für Fensterbeschreibung>);`
- Jede Eingabe wird als String gespeichert!

Eingaben

- Werte können auch über die Konsole übernommen werden
- Dafür Verwendung der Zeilen
 - `import java.util.Scanner;`
 - `Scanner sc = new Scanner(System.in);`
 - `int test = sc.nextInt();`

Verzweigungen

If - Bedingung

- Eine Anweisung wird ausgeführt, wenn eine Bedingung erfüllt ist
- Erfüllt heißt „true“ in einem boolean oder durch Vergleichsoperatoren
- Die Bedingung lässt sich mit logischen Operatoren zusammensetzen

■ Syntax:

```
■ if (<Bedingung>
{
    <Anweisung>;
}
```

■ Beispiel:

```
■ int z1 = 2, z2 = 5;
  if (z1 < z2)
  {
      System.out.print(z1 +
          " ist kleiner als " + z2);
  }
```

If - Bedingung - Beispiel

- Durch eine if-Bedingung soll ausgedrückt werden, dass ein Text in der Konsole ausgegeben wird, wenn bei den zuvor initialisierten int Werten zahl1, zahl2, zahl3 die erste Zahl größer als die zweite oder die zweite größer als die dritte ist
- **Beispiel:**
 - ```
if ((zahl1>zahl2) || (zahl2>zahl3))
{
 System.out.println("Die Bedingung wurde mit den Zahlen "+zahl1+ "
 "+zahl2+ " und "+zahl3+ " erfüllt.");
}
```

# If – else Bedingung

- Eine if-Bedingung kann mit „else“ erweitert werden
- Der Code von else wird ausgeführt, wenn die if-Bedingung nicht erfüllt wurde
- durch „else if“ kann noch weiter verzweigt werden

## ■ Syntax:

```
■ if (<Bedingung>
{
 <Anweisung>;
}
else if (<Bedingung>)
{
 <Anweisung>;
}
else
{
 <Anweisung>;
}
```

# If - else Bedingung - Beispiel

- Durch eine else-Bedingung soll das letzte Beispiel erweitert werden. Es soll abgefragt werden ob zahl3 kleiner als zahl1 ist. Wenn auch diese Bedingung nicht erfüllt ist, soll dies durch eine Textausgabe vermerkt werden

- **Beispiel:**

```
■ if ((zahl1>zahl2) || (zahl2>zahl3))
{
 System.out.println("Die Bedingung wurde mit den Zahlen "+zahl1+", "+zahl2+" und "+zahl3+" erfüllt.");
}
else if (zahl3<zahl1)
{
 System.out.println("Die dritte Zahl ist kleiner als die erste.");
}
else
{
 System.out.println("Die Zahlen "+zahl1+", "+zahl2+" und "+zahl3+" erfüllen keine der Bedingungen.");
}
```